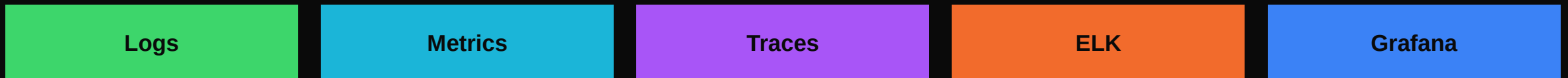
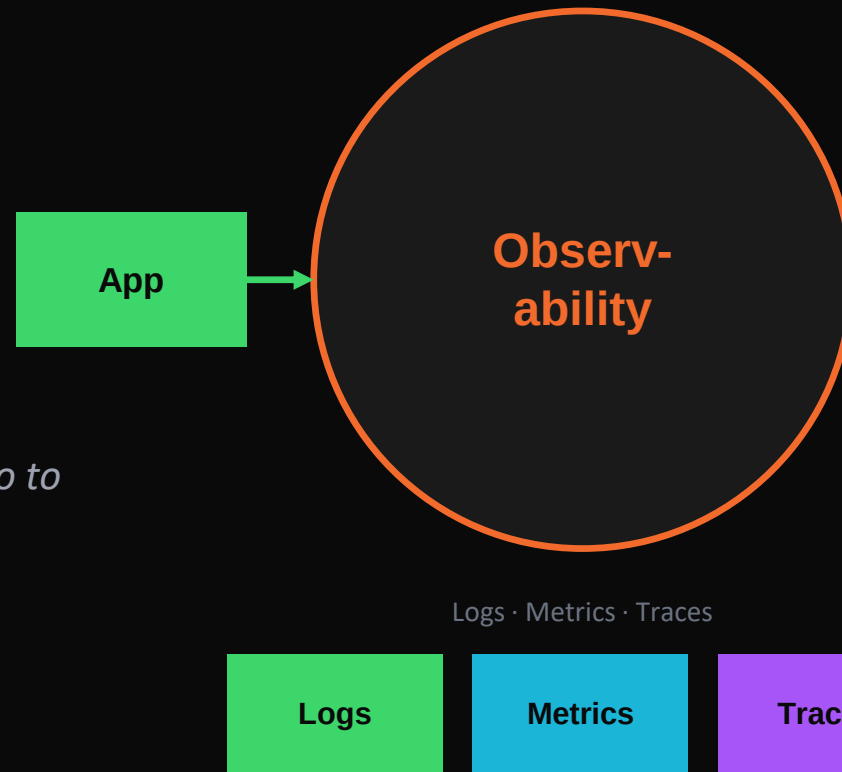


Logging & Monitoring

Centralized logging, structured logs, distributed tracing, and an intro to observability.



AGENDA

What We'll Cover

01

Why Observability

The three pillars and the failed-request problem

02

Logs & Log Levels

Structured logging and severity, done right

03

Centralized Logging

Shipping everything into the ELK Stack

04

Metrics & Alerting

Four golden signals, Grafana, and good alerts

05

Distributed Tracing

One traceId across services and message queues

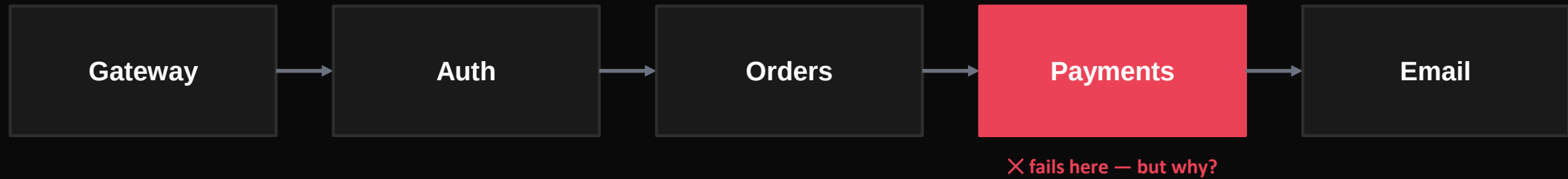
06

The Lab

Spin up ELK with Docker Compose

A Request Fails. Where Do You Look?

One user click touches five services. When it breaks, scattered logs won't save you.



WHAT HAPPENED?

Logs

The story of individual events — the actual error message and context.

HOW BAD / HOW OFTEN?

Metrics

Numbers over time that quantify system health and trends.

WHERE IN THE CHAIN?

Traces

Follow one request across every service to find the slow hop.

Log Levels: A Severity Dial

Run production at INFO; turn the dial up to DEBUG only when chasing a bug.

ERROR	Something failed and needs attention now	<i>Payment gateway timeout</i>
WARN	Unexpected, but the system recovered	<i>Retry #2 succeeded</i>
INFO	Normal business events worth recording	<i>Order #4471 placed</i>
DEBUG	Detailed diagnostics — off in production	<i>Cache key = user:5:cart</i>
TRACE	Very fine-grained, rarely enabled	<i>Entered validate()</i>

Structured Logs: Write for Machines

The single most important habit in this module — it makes every later tool possible.

✗ UNSTRUCTURED

User 5 failed login at 10pm

→ *grep + regex + guesswork*

✓ STRUCTURED (JSON)

```
{ "level": "ERROR", "service": "auth",  
  "userId": 5, "event": "login_failed",  
  "traceId": "abc-123" }
```

→ *filter, aggregate, alert in one query*

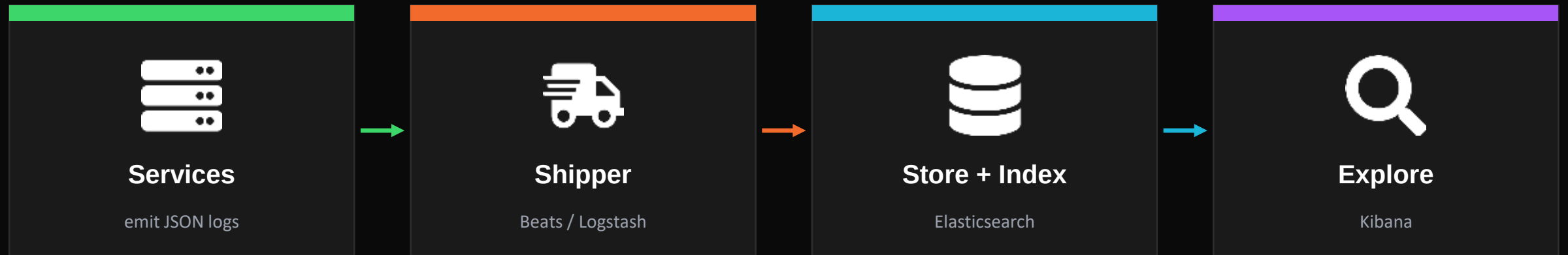


Same fields, every service. A consistent schema (timestamp, level, service, traceId) lets you ask "show all ERRORS for service=auth in the last hour" across the whole system.

CENTRALIZED LOGGING

Stop SSH-ing Into Servers

Containers die and take their logs with them. Ship every line to one searchable place.



This is the ELK / Elastic Stack — the most common open-source logging pipeline. The "B" in shipping is Beats.

The Four Golden Signals

Logs are events; metrics are numbers over time. Watch these four and you catch most problems.

Latency

How long requests take

Traffic

How much demand you're getting

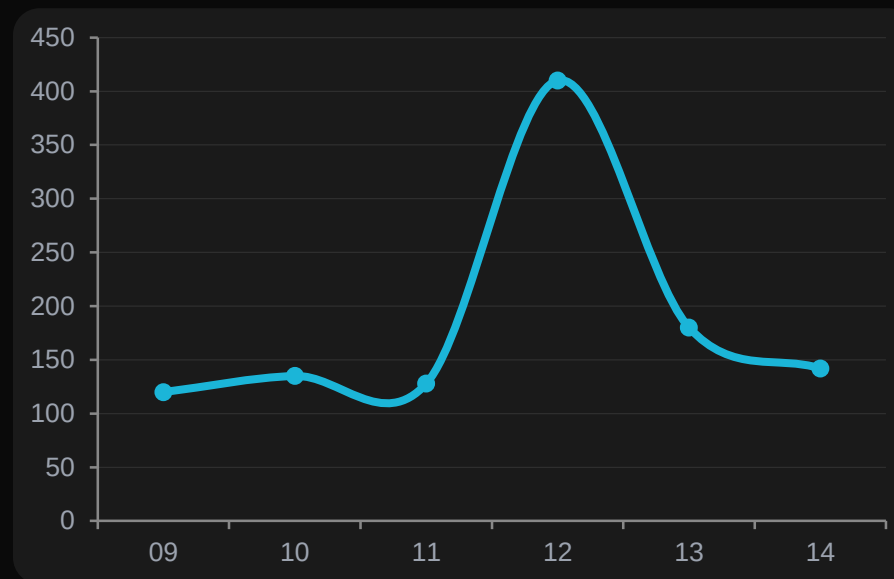
Errors

Rate of failed requests

Saturation

How full your resources are

A TYPICAL LATENCY DASHBOARD



⚡ the spike at 12:00 is your alert

Alert on Symptoms, Not Noise

The hard part isn't sending alerts — it's sending only the ones a human should act on.

✓ GOOD ALERTS

- Actionable — there's a clear response
- Tied to user impact (errors, latency)
- Has a severity: page vs. ticket
- Includes context to start debugging

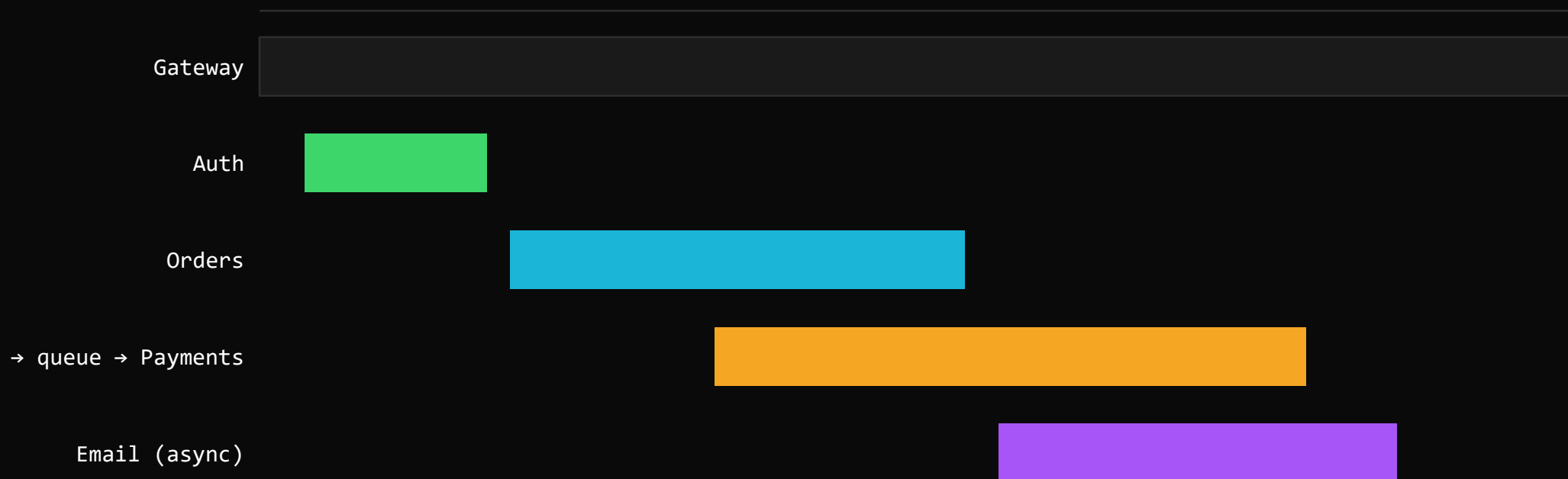
✗ ALERT FATIGUE

- Alerting on causes (CPU 80%) not symptoms
- Too sensitive — fires on every blip
- No severity — everything pages at 3am
- Result: humans ignore all of them

DISTRIBUTED TRACING

One traceId, the Whole Journey

A trace stitches together everything one request touches — including hops through message queues.



traceId "abc-123" propagates through every hop — even across the message broker.

Tools: OpenTelemetry (standard) · Jaeger · Zipkin



Hands-On Lab

Spin Up ELK With Docker Compose

Three containers, one file. You'll ship a log line and find it in Kibana.

docker-compose.yml

Save this, then run docker compose up. Elasticsearch stores, Kibana is the UI, Logstash ingests.

```
services:
  elasticsearch:
    image: docker.elastic.co/elasticsearch/elasticsearch:8.13.0
    environment:
      - discovery.type=single-node
      - xpack.security.enabled=false
    ports: ["9200:9200"]

  kibana:
    image: docker.elastic.co/kibana/kibana:8.13.0
    ports: ["5601:5601"]
    depends_on: [elasticsearch]

  logstash:
    image: docker.elastic.co/logstash/logstash:8.13.0
    ports: ["5044:5044", "5000:5000"]
    volumes: ["/usr/share/logstash/pipeline"]
    depends_on: [elasticsearch]
```

Ship a Log Line and Find It

2 · Start the stack

```
docker compose up -d
```

Wait ~60s for Elasticsearch to go green.

3 · Send a structured log

```
echo '{"level":"ERROR","service":"auth","userId":5}' | nc localhost 5000
```

Logstash receives it on TCP 5000 and forwards to Elasticsearch.

4 · Explore in Kibana

```
open http://localhost:5601
```

Create a data view, then filter: level: ERROR AND service: auth

Key Takeaways

Structure First

JSON logs with a shared schema unlock everything else.

Centralize

Ship logs off the box — ELK is the default starting point.

Four Signals

Latency, traffic, errors, saturation — visualized in Grafana.

Alert on Impact

Page humans only for actionable, user-facing problems.

Trace Across Services

Propagate one traceId; OpenTelemetry is the standard.

Correlate via traceId

The trace says where; the logs say why. The ID joins them.